Group MLR-21 WingA4_771 Major Assignment 1-2

Nitin Maheshwari (230711)
nitinm23@iitk.ac.in

Vidit Agarwal (231148)
viditag23@iitk.ac.in

Jeetraj Gupta (230507)
jeetrajg23@iitk.ac.in

Aaryan Tiwari (230023)
aaryant23@iitk.ac.in

Yugan Jain (231202)
yuganjain23@iitk.ac.in

1 Problem Abstract

Problem 1.1:

This problem requires building a **semi-parametric regression model** to predict staff members (y) based on video length (x) and popularity/difficulty factors ($\mathbf{z}$). The model structure is $y \approx w(\mathbf{z}) \cdot x + b$, where the non-linear term $w(\mathbf{z})$ is handled using a high-dimensional kernel method. The key task is to design a **custom kernel** $K(\mathbf{p}, \mathbf{p}')$ that integrates this unique semi-parametric form, allowing the model to capture the complex dependency on $\mathbf{z}$.

Problem 1.2:

This problem models the delay recovery of a **k-bit XOR Arbiter PUF**. The response $\mathbf{r}$ is modeled using the challenge's Kronecker product and a delay vector $\mathbf{v}$ derived from the original delays. The objective is to invert this linear model to estimate the 256 effective delay components $\mathbf{v}$ from challenge-response pairs. This estimation must adhere to the constraint that the actual physical delays are non-negative real numbers, utilizing 10 trained linear models for the estimation process.

2 Data Description

Problem 1.1:

The dataset provides **4000 training** and **1000 test points**. Each record contains three attributes:

- A **scalar** x (video length).
- A **vector** $\mathbf{z} = [z_1, z_2]^\top$ (audience interest z_1, difficulty z_2), with $z_i \in [0, 1]$.
- The target **scalar** y (required staff members).

This structure supports the semi-parametric model by defining distinct linear (x) and non-linear ($\mathbf{z}$) input components.

Problem 1.2:

The data consists of **10 distinct linear models**, each trained on 1089 challenge-response pairs for a $k = 32$ bit PUF. The models output 1089 estimated delays (w_i), which are real numbers between -1 and $+1$. Crucially, the actual, non-negative, real-number delays used to generate the training data are provided as a separate list of **256 true delay values** for each of the 10 models.

Preprint. Under review.

3 Tasks

3.1 Problem 1.1

3.1.1 Task 1: Derivation of the Semi-Parametric Kernel $\tilde{K}$

Problem Statement: Suppose the kernel that correctly describes the staff hiring problem is the polynomial kernel with degree d and coefficient c, i.e., $y = \mathbf{p}^\top \phi(\mathbf{z}) \cdot x + b$ where $\phi(\mathbf{z_1})^\top \phi(\mathbf{z_2}) = K(\mathbf{m}, \mathbf{n}) = (\mathbf{z_1}^\top \mathbf{z_2} + c)^d$. What would be the corresponding kernel $\tilde{K}$ that solves the semi-parametric regression problem? In other words, if the feature map ψ converts the semi-parametric problem into a purely non-parametric problem by assuring that $\tilde{\mathbf{p}}^\top \psi(x, \mathbf{z}) = \mathbf{p}^\top \phi(\mathbf{z}) \cdot x + b$ then what is the value of $\tilde{K}((x_1, \mathbf{z_1}), (x_2, \mathbf{z_2})) = \psi(x_1, \mathbf{z_1})^\top \psi(x_2, \mathbf{z_2})$?

Give detailed mathematical derivation. The kernel $\tilde{K}$ must take in two pairs of the form $(x_1, \mathbf{z_1}), (x_2, \mathbf{z_2})$ and output a similarity value. (40 marks)

Objective: We aim to convert the semi-parametric model $y = \mathbf{p}^\top \phi(\mathbf{z}) \cdot x + b$ into a standard non-parametric form $y = \tilde{\mathbf{p}}^\top \psi(x, \mathbf{z})$ solvable by **sklearn.kernel_ridge**. We need to derive the kernel function $\tilde{K}((x_1, \mathbf{z}_1), (x_2, \mathbf{z}_2))$ corresponding to the feature map ψ.

Derivation:

1. **Reformulating the Model:** We can rewrite the original equation to group the learnable parameters (the vector $\mathbf{p}$ and the scalar bias b) into a single vector.

$$y = \mathbf{p}^\top (\phi(\mathbf{z}) \cdot x) + b \cdot 1$$

Let us define an augmented weight vector $\tilde{\mathbf{p}}$ and an augmented feature map $\psi(x, \mathbf{z})$ in the new Hilbert space $\tilde{\mathcal{H}}$:

$$\tilde{\mathbf{p}} = \begin{bmatrix} \mathbf{p} \\ b \end{bmatrix}, \quad \psi(x, \mathbf{z}) = \begin{bmatrix} x \cdot \phi(\mathbf{z}) \\ 1 \end{bmatrix}$$

Taking the inner product in this space yields the original model:

$$\tilde{\mathbf{p}}^\top \psi(x, \mathbf{z}) = \mathbf{p}^\top (x\phi(\mathbf{z})) + b \cdot 1 = \mathbf{p}^\top \phi(\mathbf{z}) \cdot x + b$$

2. **Deriving the Kernel Function:** The kernel $\tilde{K}$ is defined as the inner product of the feature maps for two data points i and j:

$$\tilde{K}((x_i, \mathbf{z}_i), (x_j, \mathbf{z}_j)) = \langle \psi(x_i, \mathbf{z}_i), \psi(x_j, \mathbf{z}_j) \rangle_{\tilde{\mathcal{H}}}$$

Substituting our definition of ψ:

$$= \left\langle \begin{bmatrix} x_i \phi(\mathbf{z}_i) \\ 1 \end{bmatrix}, \begin{bmatrix} x_j \phi(\mathbf{z}_j) \\ 1 \end{bmatrix} \right\rangle$$

The inner product of concatenated vectors is the sum of the inner products of their components:

$$= \langle x_i \phi(\mathbf{z}_i), x_j \phi(\mathbf{z}_j) \rangle_{\mathcal{H}} + \langle 1, 1 \rangle_{\mathbb{R}}$$

Since x_i, x_j are scalars, they can be pulled out of the inner product:

$$= x_i x_j \langle \phi(\mathbf{z}_i), \phi(\mathbf{z}_j) \rangle_{\mathcal{H}} + 1$$

3. **Final Form:** We are given that the kernel for the feature map ϕ is a polynomial kernel $K(\mathbf{z}_i, \mathbf{z}_j) = (\mathbf{z}_i^\top \mathbf{z}_j + c)^d$. Substituting this back into the equation:

$$\tilde{K}((x_i, \mathbf{z}_i), (x_j, \mathbf{z}_j)) = x_i x_j (\mathbf{z}_i^\top \mathbf{z}_j + c)^d + 1$$

This kernel $\tilde{K}$ allows us to use Kernel Ridge Regression to learn both the non-linear dependency on $\mathbf{z}$ and the bias term b simultaneously.

3.1.2 Task 2: Hyperparameter Tuning Results

Problem Statement: Which value of the degree d and coefficient c did you find to work optimally on the semi-parametric regression data provided to you? Show charts/graphs/tables showing various combinations of c, d that you tried as hyperparameters and the corresponding test results for those hyperparameter values. The test results must be reported in terms of R^2 score (the `score` method available with `kernel_ridge` regressor objects can be used to calculate the R^2 score on test data – see validation code). (20 marks)

Optimal Hyperparameters: Based on our grid search on the provided data, we found the optimal hyperparameters to be:

- **Degree (d):** 2
- **Coefficient (c):** 20

Table 1: Hyperparameter Grid Search Results (Degree vs. Coef0 vs. R^2)

Degree	Coef0	R^2 Score	Degree	Coef0	R^2 Score	Degree	Coef0	R^2 Score
1	1	0.971833	2	25	0.972029	4	15	0.971860
1	5	0.971833	2	30	0.972029	4	20	0.971860
1	10	0.971833	3	1	0.971962	4	25	0.971847
1	15	0.971833	3	5	0.971962	4	30	0.971868
1	20	0.971833	3	10	0.971962	5	1	0.971846
1	25	0.971833	3	15	0.971962	5	5	0.971846
1	30	0.971833	3	20	0.971962	5	10	0.971847
2	1	0.972029	3	25	0.971961	5	15	0.971836
2	5	0.972029	3	30	0.971963	5	20	0.971659
2	10	0.972029	4	1	0.971860	5	25	0.971532
2	15	0.972029	4	5	0.971860	5	30	0.938291
2	**20**	**0.972029**	4	10	0.971861			

Performance Analysis: The optimal parameters yielded a Test R^2 score of: **0.972**.

Visualizations: Below are the plots showing the performance variation across different hyperparameter combinations.

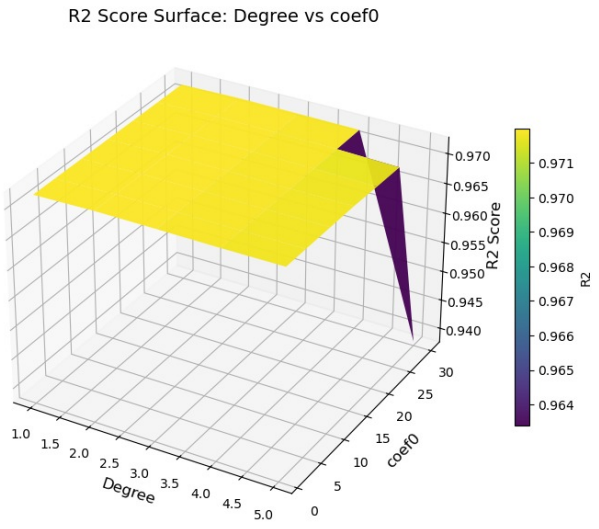

Figure 1: Degree vs. Coef0 vs. R^2 Score

Table 2: Subset of Grid Search Results

Degree (d)	Coef0 (c)	Alpha	Validation R^2 Score
1	1	1.0	0.971833
2	1	1.0	0.972029
2	5	1.0	0.972029
3	1	1.0	0.971962

3.2 Problem: 1.2

3.2.1 Task 4: Method for Delay Recovery

Problem Statement: Outline a method to take a 1089-dimensional linear model corresponding to a XOR arbiter PUF and produce 256 non-negative delays that generate the same linear model. This method should outline the equations that govern the model generation process of converting $32 \times 4 \times 2 = 256$ delays into a 1089-dimensional linear model. Then show how to invert these equations. This could be done, for example, by posing it as an (constrained) optimization problem or other ways. (40 marks)

Methodology:

Step 1: De-Kroneckerization (Matrix Factorization) The relationship $\mathbf{w} = \mathbf{u} \otimes \mathbf{v}$ implies that if we reshape the vector $\mathbf{w}$ into a matrix $\mathbf{W} \in \mathbb{R}^{33\times 33}$, then $\mathbf{W} = \mathbf{u}\mathbf{v}^\top$. This is a Rank-1 matrix. To recover $\mathbf{u}$ and $\mathbf{v}$, we perform Singular Value Decomposition (SVD) on $\mathbf{W}$:

$$\mathbf{W} = U\Sigma V^\top$$

We approximate $\mathbf{W}$ using the largest singular value σ_1 and corresponding singular vectors $\mathbf{u}_{svd}, \mathbf{v}_{svd}$:

$$\mathbf{u} = \sqrt{\sigma_1} \cdot \mathbf{u}_{svd}, \quad \mathbf{v} = \sqrt{\sigma_1} \cdot \mathbf{v}_{svd}$$

Note: We check the sign of the reconstruction against $\mathbf{W}$ to resolve sign ambiguity (since $(-\mathbf{u}) \otimes (-\mathbf{v}) = \mathbf{u} \otimes \mathbf{v}$).

Step 2: Inverting the Single Arbiter PUF Model For a single Arbiter PUF (e.g., model $\mathbf{u}$), the weights are related to the delays via intermediate parameters α and β:

$$\begin{aligned} u_0 &= \alpha_0 \\ u_i &= \alpha_i + \beta_{i-1} \quad \text{for } 1 \le i \le 31 \\ u_{32} &= \beta_{31} \end{aligned}$$

This system has 33 equations but 64 unknowns ($\alpha_0 \dots \alpha_{31}$ and $\beta_0 \dots \beta_{31}$). Since the problem allows *any* valid set of delays, we fix the free variables to simplify the solution. We set $\beta_0 = \beta_1 = \cdots = \beta_{30} = 0$. This forces:

$$\alpha_i = u_i \quad (\text{for } 1 \le i \le 31), \quad \alpha_0 = u_0, \quad \beta_{31} = u_{32}$$

Step 3: Recovering Non-Negative Delays The parameters α, β are defined by physical delays p_i, q_i, r_i, s_i:

$$\alpha_i = \frac{p_i - q_i + r_i - s_i}{2}, \quad \beta_i = \frac{p_i - q_i - r_i + s_i}{2}$$

Let $X_i = p_i - q_i$ and $Y_i = r_i - s_i$. Then:

$$X_i = \alpha_i + \beta_i, \quad Y_i = \alpha_i - \beta_i$$

To ensure non-negative delays ($p, q, r, s \ge 0$), we use the absolute value logic:

- If $X_i > 0$, set $p_i = X_i, q_i = 0$. Else, set $p_i = 0, q_i = -X_i$.
- If $Y_i > 0$, set $r_i = Y_i, s_i = 0$. Else, set $r_i = 0, s_i = -Y_i$.

This procedure is repeated for both vector $\mathbf{u}$ (PUF 1) and vector $\mathbf{v}$ (PUF 2) to obtain all 256 delays.

References

[1] Scikit-learn developers, *Scikit-learn: Machine Learning in Python*. Available at: https://scikit-learn.org/

[2] Kevin P. Murphy, *Machine Learning: A Probabilistic Perspective*. MIT Press, 2012. (Chapter on Kernel Methods)

[3] Andrew Ng, CS229 Lecture Notes on Support Vector Machines and Kernels, Stanford University, 2018.

[4] S. Devadas and G. E. Suh, "PUFs: Physical Unclonable Functions," MIT Course 6.858 Lecture Notes, 2010.